Assignment 4

Q1. Docker Compose: Nginx Static Website

Question

Create a Docker Compose setup to run an Nginx container that serves a custom HTML page.

Requirements

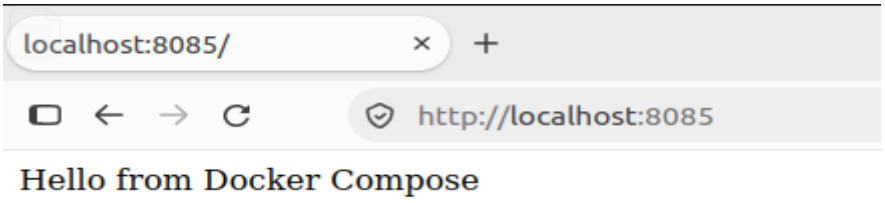
1. Create a folder named website
2. Create a file index.html with content:

Hello from Docker Compose

3. Use Docker Compose to:
   - Run an Nginx container
   - Map port 8080 to 80
   - Mount website folder to /usr/share/nginx/html
4. Open browser and verify output

```
root@server-virtual-machine:~/Assignment4# mkdir -p nginx/website
root@server-virtual-machine:~/Assignment4# cd nginx
root@server-virtual-machine:~/Assignment4/nginx# echo "Hello from Docker Compose" > we
bsite/index.html
root@server-virtual-machine:~/Assignment4/nginx#
```

```
  GNU nano 6.2                        docker-compose.yml *
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./website:/usr/share/nginx/html
root@server-virtual-machine:~/Assignment4/nginx# nano docker-compose.yml
root@server-virtual-machine:~/Assignment4/nginx# docker compose up -d --build
[+] up 1/1
 ✔ Container nginx-web-1 Started
root@server-virtual-machine:~/Assignment4/nginx#
```



Q2. Docker Compose: Python Script Container

Question

Run a simple Python script using Docker Compose.

Requirements

1. Create a file app.py: print("Hello from Python Container")
2. Create a Dockerfile to run this script
3. Use Docker Compose to run this container
4. Container should execute script and exit

```
root@server-virtual-machine:~/Assignment4# mkdir python
root@server-virtual-machine:~/Assignment4# cd python
root@server-virtual-machine:~/Assignment4/python# nano app.py
root@server-virtual-machine:~/Assignment4/python#
```

```
  GNU nano 6.2                                    app.py *
print("Hello from the Python Container")
```

```
  GNU nano 6.2                                  Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY app.py /app
CMD ["python", "app.py"
```

```
  GNU nano 6.2                            docker-compose.yml *
services:
  python-app:
    build: .
```

```
root@server-virtual-machine:~/python# docker compose up --build
[+] Building 40.7s (11/11) FINISHED
 => [internal] load local bake definitions                                 0.0s
 => => reading from stdin 490B                                             0.0s
 => [internal] load build definition from Dockerfile                       0.0s
 => => transferring dockerfile: 112B                                       0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim       21.0s
 => [auth] library/python:pull token for registry-1.docker.io              0.0s
 => [internal] load .dockerignore                                          0.0s
 => => transferring context: 2B                                            0.0s
 => [1/3] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c   16.9s
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7   0.0s
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c 251B / 251B   0.3s
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96 29.86MB / 29.86MB    15.0s
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3a 12.12MB / 12.12MB   8.6s
 => => sha256:3eb77115150676568f88e2957181b8f7e3014e797f 1.29MB / 1.29MB   6.7s
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c   1.1s
 => => extracting sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8c   0.1s
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b   0.6s
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c5   0.0s
 => [internal] load build context                                          0.0s
 => => transferring context: 79B                                           0.0s
 => [2/3] WORKDIR /app                                                     0.4s
 => [3/3] COPY app.py .                                                    0.1s
 => exporting to image                                                     2.1s
 => => exporting layers                                                    0.1s
 => => exporting manifest sha256:751ff7c19d729793f6afa9d3e4f6749ed8f95c5   0.0s
 => => exporting config sha256:d1e83f99ada229ab503a08aeaf441e7ada9a3eaf4   0.0s
 => => exporting attestation manifest sha256:af230a90c2c11a1ebc008823d04   0.0s
 => => exporting manifest list sha256:aee98481dad3cb8f22c7f638af8407c184   0.0s
 => => naming to docker.io/library/python-python-app:latest                0.0s
 => => unpacking to docker.io/library/python-python-app:latest             1.9s
 => resolving provenance for metadata file                                 0.0s
[+] up 3/3
```

```
[+] up 3/3
 ✔ Image python-python-app       Built                                40.8s
 ✔ Network python_default        Created                               0.1s
 ✔ Container python-python-app-1 Created                               0.5s
Attaching to python-app-1
python-app-1  | Hello from Python Container
python-app-1 exited with code 0
root@server-virtual-machine:~/python#

w Enable Watch   d Detach
```

Q3. Docker Compose: MySQL with Volume

Question

Create a MySQL container using Docker Compose with persistent storage.

Requirements

1. Use MySQL image
2. Set:
   - root password
   - database name
3. Create a named volume
4. Start container and:
   - Enter MySQL using docker exec
   - Create table test
5. Restart container and verify data still exists

```
root@server-virtual-machine:~/Assignments# mkdir Assignment4 && cd Assignment4
root@server-virtual-machine:~/Assignments/Assignment4# nano docker-compose.yml
root@server-virtual-machine:~/Assignments/Assignment4#
```

```
  GNU nano 6.2                          docker-compose.yml
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: mysecretpassword
      MYSQL_DATABASE: my_database
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

```
root@server-virtual-machine:~/Assignments/Assignment4# nano docker-compose.yml
root@server-virtual-machine:~/Assignments/Assignment4# docker compose up -d
[+] up 17/17
 ✔ Image mysql:8.0                Pulled                                    41.7s
 ✔ Network assignment4_default    Created                                    0.1s
 ✔ Volume assignment4_mysql_data  Created                                    0.0s
 ✔ Container assignment4-db-1     Started                                    8.7s
root@server-virtual-machine:~/Assignments/Assignment4# docker compose exec -it db mysql -u root -pmysecretpassword
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database my_database;
```

```
mysql> use my_database;
Database changed
mysql> CREATE TABLE test (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO test (name) VALUES ('Persistence Test');
Query OK, 1 row affected (0.01 sec)

mysql> select * from test;
+----+------------------+
| id | name             |
+----+------------------+
|  1 | Persistence Test |
+----+------------------+
1 row in set (0.00 sec)

mysql> exit
Bye
root@server-virtual-machine:~/Assignments/Assignment4#
```

```
root@server-virtual-machine:~/Assignments/Assignment4# docker compose restart db
[+] restart 0/1
 ⠿ Container assignment4-db-1 Restarting                                    0.9s
root@server-virtual-machine:~/Assignments/Assignment4# docker compose exec -it db mysql -u root -pmysecretpassword -e "USE my_database; SELECT * FROM test;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+------------------+
| id | name             |
+----+------------------+
|  1 | Persistence Test |
+----+------------------+
root@server-virtual-machine:~/Assignments/Assignment4#
```

Q4. Docker Compose: One Volume with 3 Nginx Containers

Question

Create a Docker Compose setup using **three Nginx containers** that share a single volume but display different HTML content.

Requirements

1. Create one named volume:

```
shared-data
```

2. Create three Nginx containers:

```
nginx1
nginx2
```

```
nginx3
```

3. Attach the same volume to all containers at:

```
/usr/share/nginx/html
```

4. Each container should display different HTML content:

```
nginx1 → Hello from Container 1
nginx2 → Hello from Container 2
nginx3 → Hello from Container 3
```

5. Expose each container on different ports:

```
nginx1 → 8081
nginx2 → 8082
nginx3 → 8083
```

6. Ensure all three containers run simultaneously using Docker Compose.
7. Open browser and verify:

```
http://localhost:8081
http://localhost:8082
http://localhost:8083
```

Each port should show different content.

```
root@server-virtual-machine:~/Assignments/Assignment4# mkdir shared-volume-nginx
root@server-virtual-machine:~/Assignments/Assignment4# cd shared-volume-nginx
root@server-virtual-machine:~/Assignments/Assignment4/shared-volume-nginx# nano docker-compose.yml
root@server-virtual-machine:~/Assignments/Assignment4/shared-volume-nginx#
```

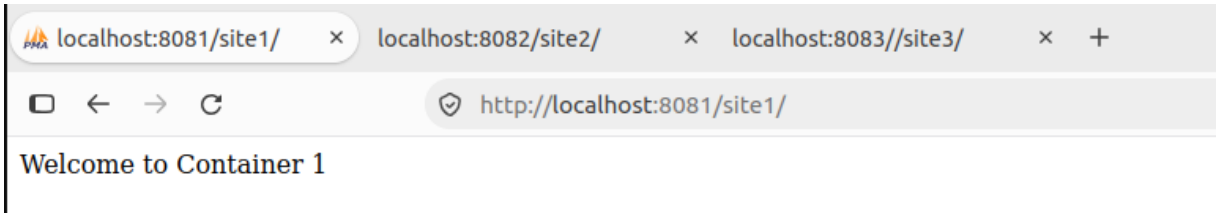
```
 GNU nano 6.2                                  docker-compose.yml
services:
  nginx1:
    image: nginx:latest
    ports:
      - "8081:80"
    volumes:
      - shared_html:/usr/share/nginx/html
    # Creating a unique directory subfolder on start to separate content
    command: /bin/sh -c "mkdir -p /usr/share/nginx/html/site1 && echo 'Welcome to Container 1' > /usr/share/nginx/html/site1/index.html && nginx -g 'daemon off;'"

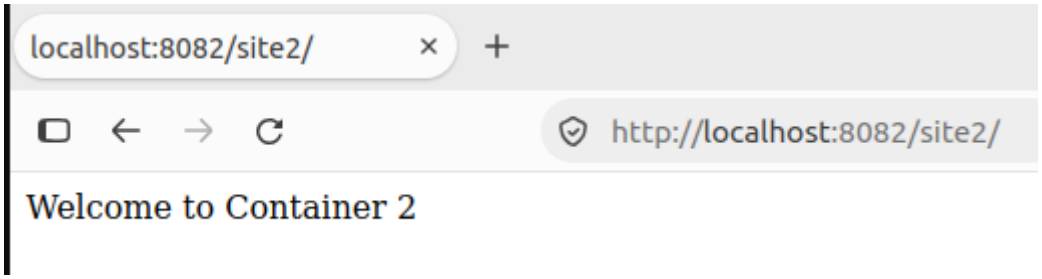
  nginx2:
    image: nginx:latest
    ports:
      - "8082:80"
    volumes:
      - shared_html:/usr/share/nginx/html
    command: /bin/sh -c "mkdir -p /usr/share/nginx/html/site2 && echo 'Welcome to Container 2' > /usr/share/nginx/html/site2/index.html && nginx -g 'daemon off;'"

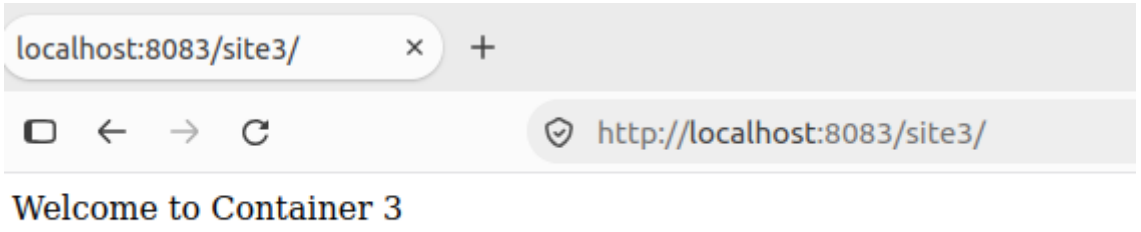
  nginx3:
    image: nginx:latest
    ports:
      - "8083:80"
    volumes:
      - shared_html:/usr/share/nginx/html
    command: /bin/sh -c "mkdir -p /usr/share/nginx/html/site3 && echo 'Welcome to Container 3' > /usr/share/nginx/html/site3/index.html && nginx -g 'daemon off;'"

volumes:
  shared_html:
```

```
root@server-virtual-machine:~/Assignments/Assignment4/shared-volume-nginx# docker compose up -d
[+] up 5/5
 ✔ Network shared-volume-nginx_default    Created
 ✔ Volume shared-volume-nginx_shared_html Created
 ✔ Container shared-volume-nginx-nginx3-1 Started
 ✔ Container shared-volume-nginx-nginx2-1 Started
 ✔ Container shared-volume-nginx-nginx1-1 Started
root@server-virtual-machine:~/Assignments/Assignment4/shared-volume-nginx# S
```







Q. 5 Write a steps for kubernetes Installation (Minikube)

1. Install kubectl (Kubernetes CLI)
kubectl is required to interact with your Kubernetes cluster.

2. Install Minikube
Choose your OS and install Minikube using one of the official methods.
Linux (example: binary download)

curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64

Windows

- Download the installer from the official Minikube site.
- Installer automatically adds Minikube to PATH.

macOS

- Use Homebrew or download the binary from the release page (per Minikube docs).

Let your user run Docker (once)

```
sudo usermod -aG docker $USER && newgrp docker
```

Start the cluster (first run takes a few minutes)

```
minikube start --driver=docker
```

3. Start Minikube (Create Kubernetes Cluster)

```
minikube start
```

This creates a single-node Kubernetes cluster locally.

4. Verify Cluster Status

```
minikube status
```

You should see components like host, kubelet, apiserver running.

Check the node is Ready

```
minikube kubectl -- get nodes
```

Create a kubectl shortcut

```
echo 'alias kubectl="minikube kubectl --"' >> ~/.bashrc
```

Load the shortcut

```
source ~/.bashrc
```

5. Verify kubectl Connectivity

```
kubectl version --client
kubectl get nodes
```
